Circles Score Group Audit

Gnosis Ltd - Report by Côme du Crest

2026-05-28

Table of contents

- Table of contents
- Circles Score Group Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [High][Unexploitable] Policy can be tricked by sending same collateral twice
  - Optimisations and miscellaneous

Circles Score Group Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes new contracts of branch `feat/score-based-group` of repo `aboutcircles/circles-groups`. The audited commit of the branch is `0x70c0ce0`.

The contracts have already been audited as of commit `0x9d38f5a`.

The contracts have been updated to remove unused code identified after the audit and have been reviewed as of commit `0x02ba07b`

Context

The new branch add the `ScoreGroup` and related contracts. This new group uses off-chain scores to allow minting a ratio of newly minted users CRC towards the group CRC.

A description of the architecture has been provided `in Notion`.

There are typically two flows to interact with the score group:

- through the mint router to migrate existing CRCs using the regular flow matrix of the Hub
- through personal mint using `groupMint()` on the Hub

In both flows, the Hub calls `_groupMint()` to verify the group's mint policy `beforeMintPolicy()` before transferring CRCs to the registered group treasury. The difference is that in the first case the router is the `sender` address for the final transfer to the group which requires that the router trusts the personal CRC while in the second case personal CRCs are directly sent to the group's treasury.

Router Mints

The Hub's `operateFlowMatrix()` function is used in a path `user -> router -> group` while an admin enabled the user's CRC for routing. When `beforeMintPolicy()` is called, the minter will be the `pathMintRouter` for the registered group. The policy will ensure that the total supply of the collateral is snapshot and accept CRCs up to the limit of snapshot amount plus the amount already minted through the personal mint flow.

The CRCs tokens will be transferred to the score treasury with `from = router`. The treasury will send all the tokens to the high score treasury.

Personal Mints

Personal mints must be wrapped in (or in the hour as) personal CRCs issuance calls in the following way:

- user calls `Policy.snapshotIssuance()` to snapshot its pending CRCs mint
- user calls `Hub.personalMint()` to mint its personal CRCs
- user calls `Hub.groupMint()` to mint score group CRCs using its newly minted CRCs

The `beforeMintPolicy()` will be called with a minter that is not the router. It will:

- ensure there is a snapshot the total supply of collateral
- validate that issuance has just been claimed and none is pending
- verify that the encoded user score is part of a sparse Merkle tree
- ensure at most a score/maxScore ratio of tokens are being minted
- reset the snapshot issuance
- update the tracked amount of personal mint registered for the collateral
- update transient storage for `keccak256(abi.encode(minter, block.timestamp))` to the corresponding collateral score

Afterwards when the treasury receives the tokens from the Hub, it will consume the score via `Policy.consumeScore()` and route tokens to either the low or high score treasury.

Timing of First Mints

After a user's CRCs have been approved for routing but before the first interaction with the policy, any CRCs issued are considered to have a perfect score and can be fully minted towards the score group tokens. I assume that users CRCs will be migrate to the score group token shortly after enabling them on the router.

Users cannot consume other's score

The function `Policy.consumeScore()` is permissionless and could be called by anyone in between the moment when the score is set and before it should be rightfully consumed.

However, the score is set in the Hub's `_groupMint()` (in case router has not been used) which calls `beforeMintPolicy()` and `safeBatchTransferFrom()` immediately after to trigger `treasury.onERC1155Received()` which consumes the score.

Demurrage calculation discrepancy

The function used to calculate a demurraged amount of CRCs based on the amount of days does not yield the same amount if computed twice on 1 day demurrage or once on 2 days demurrage. This means that if the calculation of demurrage does not occur on the same day in the Policy and the Hub the tracked amounts will be slightly off. Among the Hub and the Policy, the one that computes demurrage the most frequently will end up with the lower total balance computed.

I do not foresee the small balance discrepancy to be a problem anywhere in the Policy / Treasury.

Redeem Mechanism

To redeem score group CRCs back into personal CRCs users can simply operate a flow matrix to transfer group CRCs back to the treasury to receive the personal CRCs as each subtreasuries trust the group CRC. The treasury burns held score group CRCs upon receipt.

Status

The report has been sent to the core developer.

The sole identified issue is in practice not exploitable in the context the contracts are used. Code has been updated and reviewed to better reflect this. No issues remain.

Legal Information And Disclaimer

1. This report is based solely on the information provided by [project], with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to Gnosis, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[High][Unexploitable] Policy can be tricked by sending same collateral twice

Summary

The function `OffchainScoreBasedMintPolicy.beforeMintPolicy()` when called through router mints limits the amounted based on the amount of collateral in the treasury. If the function is called with the same collateral twice, it will perform the second limit check before the treasury received the tokens from the first mint resulting in incorrect limitations.

Vulnerability Detail

The function `beforeMintPolicy()` loops through sent collateral and computes the `current Limit` as the max accepted amount on the treasury minus the currently held amount by the treasury. It then enforces that the sent amount is below that limit:

```
    function beforeMintPolicy(
        address minter,
        address group,
        uint256[] calldata collateral,
        uint256[] calldata amounts,
        bytes calldata data
    ) external onlyHub returns (bool) {
        if (minter != pathMintRouters[group]) {
            ...
        } else {
            // migration mint branch
            address treasury = HUB.treasuries(group);
            for (uint256 i; i < collateral.length;) {
                _ensureHistoricalSupplyInitialized(group, collateral[i]);

                // check limits
                uint256 historicalSupplyOnToday = getHistoricalSupplyOnToday(
                    group, collateral[i]);
                // personal minted
                uint256 mintedAmountOnToday = getMintedAmountOnToday(group,
                    collateral[i]);
                uint256 maxLimit = historicalSupplyOnToday +
                    mintedAmountOnToday;

                uint256 treasuryBalance;
                try IScoreTreasury(treasury).balanceOfCollateral{gas: 100_000}(
                    collateral[i]) returns (
                    uint256 returnedBalance
                ) {
                    treasuryBalance = returnedBalance;
                } catch {
```

```
28                      treasuryBalance = HUB.balanceOf(treasury, collateral[i]);
29                  }
30
31                  if (treasuryBalance > maxLimit) revert CollateralLimitReached()
                        ;
32
33                  uint256 currentLimit = maxLimit - treasuryBalance;
34
35                  if (amounts[i] > currentLimit) revert
                        AmountExceedsCollateralLimit();
36                  ...
37                  unchecked {
38                      ++i;
39                  }
40              }
41          }
42          return true;
43      }
```

A collateral could have a limit of 100 and anyone holding that collateral could send twice 100 in the same transfer. The Policy would still return `true` and accept the group token mint.

Impact

Policy can be cheated to mint more group score tokens than allowed if treasury does not already hold tokens up to the limit.

Code Snippets

https://github.com/aboutcircles/circles-groups/blob/70c0ce0ba551372a89b535dfa6497a954ea7a60a/src/score-group/OffchainScoreBasedMintPolicy.sol#L349

Recommendation

Track each collateral verified in the current call to `beforeMintPolicy()` and revert if a collateral is used twice. This can be done for example by enforcing that each collateral in the `collateral[]` array is a smaller uint256 than the previous one. This would require the function to be called with ordered collateral IDs.

Response

After further investigation, it is not possible to send the same collateral twice to `beforeMintPolicy()` when using a transfer through the router.

A router transfer will go through the Hub's `operateFlowMatrix()` which calls `_effectPathTransfers()` which loops over the different type of collateral to call `_groupMint()` which thus calls `Policy.beforeMintPolicy()` with an array containing a single collateral ID.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

Useless `mstore()` in `_onlyHuman()`

The function `Score.group._onlyHuman()` uses assembly to oddly verify that an avatar is human. It writes twice at the memory position `pointer` in a row. This means the second first written value is replaced and never used the write `mstore(pointer, 0x44)` could be removed.

https://github.com/aboutcircles/circles-groups/blob/70c0ce0ba551372a89b535dfa6497a954ea7a60a/src/score-group/ScoreGroup.sol#L289

`beforeMintPolicy()` uses `maxLimit` value that is undescriptive

The function `OffchainScoreBasedMintPolicy.beforeMintPolicy()` defines `maxLimit` which represents the demurraged maximum amount of tokens that should be held in the treasury as defined by the historical personal CRC supply accepted at a perfect score plus the total amount of group CRCs minted via this collateral up till today. I find the name `maxLimit` to be undescriptive and lacking documentation.

https://github.com/aboutcircles/circles-groups/blob/70c0ce0ba551372a89b535dfa6497a954ea7a60a/src/score-group/OffchainScoreBasedMintPolicy.sol#L346